

Activity 10: Layer2 and wireless security

TGS-2023037854 | v7.0 | 13 September 2026 | 90 minutes

Goal

Map trusted infrastructure ports and untrusted edge ports to DHCP snooping, DAI, port security and wireless authentication.

Before you start

Python 3.9+, a plain text editor and this complete folder. Offline mode requires no extra packages, Internet or credentials. The compulsory core Packet Tracer construction is specified below and in topology.md; advanced extensions may need CML/IOS XE.

Evidence boundary

All JSON/CSV observations are synthetic training evidence. Passing the Python checker validates the fixture contracts only. No hardware, simulator, authentication server or API is configured or contacted.

Files

README.md, instructions.pdf, checklist.pdf, topology.md, case.json, faulty.json, reference.json, observations.csv, specimen.cfg, R1-baseline.cfg, S1-baseline.cfg

Working method

case.json is the working evidence; faulty.json resets the exercise and reference.json shows corrected synthetic evidence for self-study. Preserve your own sandbox observations separately.

Detailed step-by-step instructions

1. Construct and save the mandatory simulator topology

Start Cisco Packet Tracer 8.x or a current compatible version. File > New; use the bottom device palette, drag these models into the workspace and rename them via Config > Display Name: 1 x Switch2960 (S1), 1 x Router2911 (R1), 1 x Server-PT (DHCP), 2 x PC-PT (PC1,PC2). Choose Connections > Copper Straight-Through for the table unless otherwise stated. Switch-to-switch links use Copper Cross-Over or Automatically Choose Connection Type. Click the first device and select its named interface; click the peer and select its named interface. Wait for links to initialize. Save as activity-start.pkt.

Device/port A	Device/port B	Cable
PC1 FastEthernet0	S1 FastEthernet0/1	Straight-through
PC2 FastEthernet0	S1 FastEthernet0/2	Straight-through
DHCP FastEthernet0	S1 FastEthernet0/24	Straight-through
R1 GigabitEthernet0/0	S1 GigabitEthernet0/1	Straight-through

Save both activity-start.pkt and a working activity-working.pkt; use File > Save As.

2. Assign endpoint addresses and inspect interface identifiers

Set Desktop > IP Configuration > Static on each endpoint.

Endpoint	Address	Mask	Gateway	DNS
DHCP	10.10.10.50	255.255.255.0	10.10.10.1	10.10.10.50
PC1	DHCP	DHCP	DHCP	DHCP
PC2	DHCP	DHCP	DHCP	DHCP

DHCP Services > DHCP: On; pool USERS; Default Gateway10.10.10.1; DNS10.10.10.50; Start IP10.10.10.100; Mask255.255.255.0; maximum50; Add/Save. PC1/PC2 Desktop > IP Configuration > DHCP.

For routers/switches open CLI. Answer no to the initial configuration dialog and press Enter. Use the exact interface names below; a 2911 uses GigabitEthernet0/0 through 0/2, a 2960/3560 uses FastEthernet0/1 through 0/24 and GigabitEthernet0/1 through 0/2.

```
enable
show ip interface brief
```

3. Install all peer baseline configurations

Paste the following device-specific blocks into each named device CLI one device at a time. Wait for the router/switch prompt between blocks. These are complete activity configurations for the stated construction. Do not apply configuration from a different activity. When copy asks for Destination filename, press Enter. Save the Packet Tracer working file after all blocks.

```
Device R1
enable
configure terminal
hostname R1
no ip domain-lookup
interface GigabitEthernet0/0
ip address 10.10.10.1 255.255.255.0
no shutdown
exit
end
copy running-config startup-config
```

Device S1

```
enable
configure terminal
hostname S1
no ip domain-lookup
vlan 10
interface range FastEthernet0/1-2, FastEthernet0/24, GigabitEthernet0/1
    switchport mode access
    switchport access vlan 10
    no shutdown
exit
ip dhcp snooping
ip dhcp snooping vlan 10
no ip dhcp snooping information option
interface FastEthernet0/24
    ip dhcp snooping trust
exit
interface FastEthernet0/1
    switchport port-security
    switchport port-security maximum 1
    switchport port-security mac-address sticky
    switchport port-security violation restrict
exit
end
copy running-config startup-config

show running-config
show ip interface brief
copy running-config startup-config
```

4. Prove the healthy baseline with actual packets

Run every test below before introducing the fault. PC/Desktop > Command Prompt provides ping, ipconfig and arp. Use Simulation mode and Edit Filters to keep ARP, ICMP and the protocol under study; click Add Simple PDU for a ping and Capture/Forward to inspect hops. The first ping may lose one packet during ARP; repeat and record the second result. Do not mark a test passed until observed.

PC1/PC2 get 10.10.10.100+ leases; ping 10.10.10.1 succeeds. S1 show port-security interface FastEthernet0/1 displays secure-up/max1 and violation counter; show port-security address displays PC1 MAC. show ip dhcp snooping shows VLAN10 and only Fa0/24 trusted; show ip dhcp snooping binding shows clients if supported by PT. DAI/storm-control/RA Guard may be absent on the selected PT model; test core snooping and port security, then inspect extension fixture separately.

After all healthy tests pass, File > Save As > activity-start.pkt to preserve the CONFIGURED healthy baseline, then File > Save As > activity-working.pkt before introducing the fault. The initial empty construction is not the rollback baseline.

Save baseline screenshots / CLI text with device names; File > Save.

5. Introduce and diagnose the controlled fault

Save the baseline first. Apply only the following fault to the specified device. Repeat the same traffic tests, capture the first failing hop/control and compare show output with the healthy baseline.

First learn PC1 sticky MAC by pinging the gateway. Disconnect PC1 from S1 Fa0/1 and move PC2 from Fa0/2 to Fa0/1 using the same cable type; ping the gateway. Restrict-mode violation should increment and new-source traffic should be blocked while the link remains up. Next restore cabling. Separately remove DHCP trust from server-facing Fa0/24; renew a client lease and inspect rejected offers.

Retain activity-start.pkt and baseline running configurations; work in activity-working.pkt.

6. Apply the narrow repair and re-test the network

Restore PC1 to Fa0/1 and PC2 to Fa0/2. Restore ip dhcp snooping trust ONLY on Fa0/24, not endpoint ports. Preserve sticky PC1 MAC; if it was learned incorrectly, remove the exact saved sticky line and relearn the authorized host.

Repeat the baseline test matrix. Check surrounding controls as well as the repaired flow. why DHCP trust belongs toward the authorized server and why port-security restrict can block a rogue MAC without shutting the port

Save actual show output and allowed/denied traffic results; copy running-config startup-config only after successful verification.

7. Rollback and rehearse recovery

Save the repaired file as activity-verified.pkt. To undo the exercise, open the saved healthy baseline activity-start.pkt after you have saved configurations into it at Step 3, or paste the saved pre-fault running configuration into a clean topology. For the specific repair, retain the exact old command/value rather than guessing a trunk list or ACL. Repeat one positive and one relevant negative test after recovery.

File > Save As > activity-verified.pkt; close working topology; reopen saved healthy baseline.

8. Submit observed evidence and explain the forwarding mechanism

Submit the verified .pkt file, a cabling/address table, pre-fault and post-repair show outputs, screenshots of traffic tests, diagnosis and rollback notes. Explain why DHCP trust belongs toward the authorized server and why port-security restrict can block a rogue MAC without shutting the port Keep synthetic checker output separate from real simulator observations.

Record simulator version, device models and tests actually completed.

9. Complete the separately labelled extension

For DAI, on supported IOS XE/CML enable ip arp inspection vlan10 and trust only the legitimate infrastructure direction; validate binding before injecting a mismatched ARP. RA Guard is host-edge only and storm thresholds require measured baselines. Repeat the WPA2-PSK wrong-key test from Activity05 in a separate wireless copy; compare 802.1X authentication concept using the enterprise fixture. Mark unsupported PT commands evidence-only.

Record supported platform, output observed or evidence-only limitation.

Untimed appendix: independent fixture analysis

These synthetic cases supplement the allocated hands-on time and do not configure the simulator.

Appendix 1. Establish the baseline

Open activity10-layer2-wireless-security; inspect topology.md, case.json and observations.csv. Identify the controls for: Map trusted infrastructure ports and untrusted edge ports to DHCP snooping, DAI, port security and wireless authentication.

```
python3 --version
```

Appendix 2. Run the faulty evidence case

Run the checker against case.json; exit status 1 is expected at this stage. Record each FAIL label and the calculated detail.

```
python3 checker.py case.json
```

Appendix 3. Propose the repair

For the endpoint set `dhcp_trust` and `dai_trust` false, `ra_guard` true, `storm_percent` 1; restore the ARP MAC to 0011.2233.4455. Retain the infrastructure uplink trust. Explain why blanket uplink RA blocking could break legitimate SLAAC.

Open `case.json` in a plain text editor; preserve JSON types.

Appendix 4. Validate the corrected case

Save a separate working copy first. Run the checker after each narrow edit until all controls PASS; capture output as evidence. This validates fixtures only.

```
python3 checker.py case.json > evidence.txt
```

Appendix 5. Verify checker behavior

Run self-test; it must accept corrected reference evidence and reject the faulty fixture and the independent corrupted-control test.

```
python3 checker.py --self-test
```

Expected results and evidence

Corrected reference: all checks PASS, exit 0; faulty fixture: one or more FAIL, exit 1; self-test: PASS, exit 0.

Domain-specific computed fields appear in JSON detail.

Capture `evidence.txt`, corrected `case.json` and your diagnosis. For sandbox work record device/model, timestamps, actual show outputs, allowed/denied tests and rollback. Distinguish fixture results from real observations.

Troubleshooting and reset

Exit 1: inspect the named failed contract. Exit 2: restore valid JSON keys/types. Unsupported simulator commands must be documented; never invent results. Reset by copying `faulty.json` over `case.json`. Restore sandbox settings from the actual saved baseline.

Calculated corrected-fixture results

```
edge_ports: ["Gi1/0/1"]
```

```
ARP_binding_match: true
```

Required PASS controls

- untrusted endpoints
- trusted infrastructure uplink
- DAI binding matches ARP
- refresh RA guard at edge
- refresh storm threshold bounded
- enterprise AES no embedded PSK

Transition practice

Refresh extension: RA Guard and storm control protect different traffic classes. The fixed 1% threshold is an exercise requirement; production limits depend on baseline traffic and switch capabilities.

Configuration files

The complete per-device configurations are shown in the construction steps and included as named

*-baseline.cfg files. `specimen.cfg` combines those same blocks with explicit device boundaries. Apply each block only to its named device. WRT300N uses GUI settings instead of IOS CLI.

References

Cisco CCNA v1.1 exam topics; Cisco IOS XE configuration guides; Python ipaddress documentation. Exact URLs and source links are in README.md.